

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена трудового Красного Знамени федеральное государственное
бюджетное
образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Математическая кибернетика и информационные технологии

**Лабораторная работа №4
Разработка веб-приложения на Spring Boot.
Работа с данными
по дисциплине
«Информационные технологии и программирование»**

Выполнил: студент гр. БВТ2403
Толибов Р.Ш.

Руководитель:

Москва, 2026 г.

Цель работы

Научиться работать с данными в Spring Boot-приложении с использованием Spring Data JPA и Hibernate: подключать приложение к базе данных PostgreSQL, создавать JPA-сущности и репозитории, реализовывать базовые CRUD-операции, настраивать связи между таблицами, использовать транзакции, валидировать входные данные и обрабатывать ошибки.

Задачи

- Подготовить окружение: установить PostgreSQL, DBeaver, создать базу данных.
- Добавить в проект зависимости Spring Data JPA, PostgreSQL, Lombok, Validation.
- Настроить подключение к PostgreSQL через переменные окружения и application.properties.
- Создать JPA-сущности User и Notification со связью один-ко-многим.
- Создать перечисления NotificationChannel и NotificationStatus.
- Реализовать DTO для пользователя и уведомления.
- Создать репозитории UserRepository и NotificationRepository.
- Разработать CRUD-операции для пользователей и уведомлений в сервисах и контроллерах.
- Освоить методы репозитория: производные запросы, сортировка, @Query (JPQL и native).
- Изучить механизм транзакций @Transactional.
- Добавить валидацию входных DTO с помощью аннотаций Bean Validation.
- Выполнить самостоятельные задания по оптимизации кода и расширению функциональности.

Ход работы

0. Подготовка

На компьютере были установлены PostgreSQL, DBeaver и Postman. В DBeaver создано новое подключение к PostgreSQL (хост localhost, порт 5432, пользователь postgres). Через интерфейс DBeaver создана база данных с именем spring_lab3_notifications. Правильность подключения проверена через выполнение простого SQL-запроса.

1. Подключение Spring Data JPA и PostgreSQL

В проект spring-lab3-notifications (из предыдущей лабораторной работы) в файл pom.xml добавлены зависимости:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Для безопасного хранения параметров подключения создан файл .env в корне проекта:

```
DB_URL=jdbc:postgresql://localhost:5432/spring_lab3_notifications
DB_USERNAME=postgres
DB_PASSWORD=mysecretpassword
```

В VS Code в конфигурации запуска добавлены:

```
spring.datasource.url=${DB_URL}
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Параметр `ddl-auto=update` позволяет Hibernate автоматически создавать/обновлять таблицы на основе сущностей.

2. Создание сущностей и перечислений

Создана структура пакетов: `org.example.controller`, `org.example.service`, `org.example.repository`, `org.example.exception`, `org.example.model.dto`, `org.example.model.entity`, `org.example.model.enums`.

`NotificationChannel` (EMAIL, SMS, PUSH, TELEGRAM) и `NotificationStatus` (CREATED, SENT).

Сущность `User`:

```
@Entity
@Table(name = "users")
@Getter @Setter @NoArgsConstructor
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    private String email;
    private String phone;
    private String deviceToken;
    private String telegramChatId;
    private LocalDateTime createdAt;
    @OneToMany(mappedBy = "recipient", cascade = CascadeType.ALL)
    private List<Notification> notifications = new ArrayList<>();
}
```

Сущность `Notification`:

```
@Entity
@Table(name = "notifications")
@Getter @Setter @NoArgsConstructor
public class Notification {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String title;
    @Column(nullable = false, columnDefinition = "TEXT")
    private String message;
    @Enumerated(EnumType.STRING)
    private NotificationChannel channel;
    @Enumerated(EnumType.STRING)
    private NotificationStatus status;
    private LocalDateTime createdAt;
}
```

```

private LocalDateTime sentAt;
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "recipient_id", nullable = false)
private User recipient;
}

```

После запуска приложения Hibernate автоматически создал таблицы users и notifications с соответствующими колонками и внешним ключом.

3. Создание DTO и репозиториев

Для передачи данных между контроллером и сервисом созданы DTO-классы с использованием Lombok (@Builder, @Getter, @Setter и т.д.). UserDto содержит поля: name, email, phone, deviceToken, telegramChatId, createdAt.

NotificationDto содержит: title, message, channel, status, createdAt, sentAt, recipientId.

Репозитории – интерфейсы, расширяющие JpaRepository:

```

@Repository
public interface UserRepository extends JpaRepository<User, Long> { }

@Repository
public interface NotificationRepository extends JpaRepository<Notification, Long> {
    List<Notification> findByStatus(NotificationStatus status);
    List<Notification> findByChannel(NotificationChannel channel);
    List<Notification> findByRecipientId(Long recipientId);
}

```

4. Реализация CRUD для User

UserService

Сервисный класс с аннотацией @Service и внедрением UserRepository.

Реализованы методы:

- createUser(UserDto) – преобразует DTO в сущность, устанавливает createdAt, сохраняет.
- getAllUsers() – возвращает список всех пользователей.
- getUserById(Long) – находит пользователя, иначе выбрасывает исключение.
- updateUser(Long, UserDto) – обновляет поля.
- deleteUser(Long) – удаляет пользователя.

UserController

Контроллер с маппингом /users. Реализованы эндпоинты:

- POST /users/add – создание пользователя.

- GET /users/all – получение всех пользователей.
- GET /users/{id} – получение по id.
- PUT /users/{id} – обновление.
- DELETE /users/{id} – удаление.

В каждом методе контроллера сущность преобразуется в UserDto вручную (с помощью билдера).

5. Реализация CRUD для Notification

NotificationService

Сервис включает методы:

- createNotification(NotificationDto) – по recipientId находит пользователя, создаёт уведомление со статусом CREATED и текущим временем, сохраняет.
- getAllNotifications, getNotificationById, updateNotification, deleteNotification.
- Дополнительные методы поиска: getNotificationsByStatus, getNotificationsByChannel, getNotificationsByRecipientId.
- NotificationController

Эндпоинты с базовым путём /notifications:

- POST /add
- GET /all
- GET /{id}
- PUT /{id}
- DELETE /{id}
- GET /status/{status}
- GET /channel/{channel}
- GET /recipient/{recipientId}

Все методы контроллера также вручную преобразуют сущность Notification в NotificationDto.

6. Методы репозитория Spring Data JPA

В NotificationRepository добавлены дополнительные методы:

1. Запрос по двум параметрам – производный метод:

```
List<Notification> findByStatusAndChannel(NotificationStatus status, NotificationChannel channel);
```

2. Сортировка в имени метода:

```
List<Notification> findByStatusOrderByCreatedAtAsc(NotificationStatus status);
```

3. JPQL-запрос с @Query:

```
@Query("SELECT n FROM Notification n WHERE n.status = :status AND n.channel = :channel")
```

```
List<Notification> findByStatusAndChannelCustom(@Param("status") NotificationStatus status,
```

```
                                @Param("channel")
NotificationChannel channel);
```

4. Native SQL-запрос:

```
@Query(value = "SELECT * FROM notifications WHERE status = :status AND channel
= :channel", nativeQuery = true)
List<Notification> findNativeByStatusAndChannel(@Param("status") String
status, @Param("channel") String channel);
```

Эти методы были протестированы через Postman – все работают корректно.

7. Транзакции

Для демонстрации работы `@Transactional` в метод `createNotification` временно добавлена искусственная ошибка:

```
if (true) {
    throw new RuntimeException("Искусственная ошибка");
}
```

Без `@Transactional` уведомление сохранялось в БД, несмотря на исключение. После добавления аннотации `@Transactional` над методом изменения откатывались – запись не появлялась в таблице. Это подтверждает, что транзакция фиксируется только при успешном завершении метода.

8. Проверка работы приложения в Postman

С помощью Postman были выполнены следующие запросы:

POST /users/add с JSON-телом `{"name":"Иван","email":"ivan@test.com"}` – пользователь создан.

GET /users/all – получен список.

GET /users/1 – получен пользователь с `id=1`.

PUT /users/1 с обновлёнными данными – обновление успешно.

DELETE /users/1 – удаление.

POST /notifications/add – создание уведомления для существующего пользователя.

GET /notifications/all – список уведомлений.

GET /notifications/status/CREATED – фильтрация по статусу.

GET /notifications/channel/EMAIL – фильтрация по каналу.

GET /notifications/recipient/1 – уведомления конкретного получателя.

Все эндпоинты возвращали корректные ответы, данные сохранялись в PostgreSQL.

9. Валидация входных данных

В `UserDto` добавлены аннотации валидации:

```
@NotBlank(message = "Имя не должно быть пустым")
@Size(max = 100, message = "Имя не длиннее 100 символов")
private String name;
```

```
@NotBlank(message = "Email обязателен")
@email(message = "Некорректный формат email")
@Pattern(regexp = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$", message = "Email не
соответствует шаблону")
private String email;
```

В контроллере перед `@RequestBody` указана аннотация `@Valid`. При попытке создать пользователя с пустым именем или некорректным email Spring возвращает 400 Bad Request с описанием ошибок.

Самостоятельные задания

1. В контроллере создан приватный метод `mapToDto(Notification notification)`, который содержит вызов билдера. Это устранило дублирование кода в 7 методах.

2. В `UserDto` добавлена аннотация:

```
@Pattern(regexp = "^\\+?[0-9]{10,15}$", message = "Неверный формат телефона")
private String phone;
```

3. Реализован через производный метод `findByStatusAndChannel`

4. В репозиторий добавлен метод:

```
List<Notification> findAllByOrderByCreatedAtDesc();
```

5. Собственный `@Query` для поиска уведомлений по `recipientId` и статусу

```
@Query("SELECT n FROM Notification n WHERE n.recipient.id = :recipientId AND
n.status = :status")
List<Notification> findByRecipientIdAndStatus(@Param("recipientId") Long
recipientId, @Param("status") NotificationStatus status);
```

6. В `NotificationService` в метод `updateNotification` добавлена логика:

```
if (request.getStatus() == NotificationStatus.SENT && notification.getSentAt()
== null) {
    notification.setSentAt(LocalDateTime.now());
}
```

7. При вызове `createNotification` с `recipientId`, которого нет в БД, метод `userRepository.findById(...).orElseThrow()` выбрасывает `NoSuchElementException`. Глобальной обработкой исключений можно вернуть статус 404 с соответствующим сообщением.

Вывод

В ходе лабораторной работы было создано веб-приложение на Spring Boot с полноценным слоем работы с базой данных PostgreSQL. Освоены основные возможности Spring Data JPA и Hibernate: автоматическое создание таблиц, репозитории с производными методами, JPQL и native запросы.

Реализованы CRUD-операции для двух связанных сущностей (User и Notification) с использованием многоуровневой архитектуры (контроллер – сервис – репозиторий). Изучен механизм транзакций @Transactional, обеспечивающий целостность данных.

Добавлена валидация входных DTO с помощью Bean Validation, что позволяет отсеивать некорректные запросы до попадания в бизнес-логику. Выполнены самостоятельные задания по оптимизации кода (вынос маппинга), расширению методов поиска и автоматическому заполнению полей.

<https://github.com/TRuslan666/ITaP/tree/main/лаб4>